Signal Recovery from Additive White Gaussian Noise using Moving Average Filter

Submitted by:
Milind Sharma (24BEC0078)
Akshaj Sharma (24BEC0161)
Lakshya Lalan (24BEC0603)

1 Objectives

The primary objectives of this project are as follows:

- Create and investigate a combined waveform made up of many sinewaves in both time and frequency domains.
- Develop and introduce additive white Gaussian noise (AWGN) using a controlled signal-to-noise ratio (SNR) in order to assess the effect of this type of noise on the combined waveform.
- Develop and implement a moving average (MA) filter as a linear time-invariant (LTI) system to recover the combined waveform.
- Assess filter performance by calculating both improvement in SNR and mean square error (MSE).
- Study the temporal characteristics and statistical properties of the combined waveform by examining its mean, variance, and autocorrelation; conduct spectral analysis using both FFT and PSD.
- Demonstrate the efficacy of the filter in attenuating AWGN while maintaining components of the desired waveform; outline possible applications of such a filter.

2 Introduction

In today's communication and signal processing systems, there is often some level of noise interfering with the transmission and acquisition of a signal. One of the more common types of noise is called Additive White Gaussian Noise (AWGN). This results in an overall degradation in the quality and reliability of the original signal and therefore needs to be removed through recovery processes.

This project will focus on the process of recovering an original signal, which has been corrupted by AWGN through the use of a moving average (MA) filter. The original signal is specifically composed of two sinusoidal signals that clearly differ in their respective frequency rates, and the noise that will be added will be drawn from a controlled signal-to-noise ratio (SNR) to replicate real-world-type scenarios.

The moving average filter is an LTI (linear time-invariant) filter that smooths the output by averaging previous samples taken over a given period. Smoothing the output using averaging will reduce the amount of noise present in the output, while preserving the overall structure of the input signal.

Different analyses can be completed in order to evaluate how effectively the moving average filter is performing. Analyses in both the time domain and frequency domain can yield performance metrics, including SNR (signal-to-noise ratio) and MSE (mean-

squared error), as well as providing statistical parameters such as mean, variance, and autocorrelation.

Spectral analyses conducted using FFT (Fast Fourier Transform) and PSD (Power Spectral Density) demonstrate how effectively the moving average filter reduces high-frequency noise while preserving the desired components of the input signal. Furthermore, the use of the moving average filter is an application of random processes, noise modeling, spectral analysis and LTI systems.

3 Significance

The first phase of the project involves evaluating a signal through signal processing and random processes. Noise frequently interferes with signal quality; therefore, noise suppression techniques should be used when transmitting accurate data.

Although it is very straightforward, the Moving Average filter is extremely potent in terms of signal processing and utilizes minimal processing resources for real-time implementations.

As a result, this project illustrates how theoretical concepts, for example, (LTI Systems), are used by exhibiting practical applications in the real world.

To illustrate the filtering and noise effects on a signal's quality, The project will involve both time and frequency domain analysis as well as reinforcing theoretical concepts mentioned above. The impact of filtering and noise on the quality of a signal is particularly significant to the fields of communication/bio-engineering (i.e., communication systems) and embedded system design.

To conclude, the first phase of this project demonstrates a mechanism to use solely simplistic filtering methods in generating quality signals throughout the different engineering domains represented in this project.

4 Results

The performance of the Moving Average filter was evaluated using both qualitative and quantitative analysis.

4.1 Signal and Noise Characteristics

The composite signal was generated with two sinusoidal components (5 Hz; am= 1.0) and (10 Hz; am= 0.5) and sampled at 1000 Hz (i.e., 1000 samples for durations of 1 second). The composite signal has been affected by additive white Gaussian noise (AWGN) added at -5 dB SNR (i.e., 0.4446 standard deviations). This indicates substantial corruption in the composite signal due to noise.

4.2 Filter Performance Metrics

SNR (Signal to Noise Ratio) & Mean Squared Error (MSE) were used for evaluation of the 20-point Moving Average filter's performance (i.e., in relation to the original signal).
SNR at input = 4.95dB; SNR at output = 8.24dB (gain = 3.29dB).
MSE of noisy signal = 0.199903 before filtering, reduced to 0.093735 after filtering, an improvement of 53.1

4.3 Time-Domain Analysis

In the time domain we see how adding and removing noise changes the signal. The noisy signal experiences many variations, while the filtered version of that signal shows much

lower levels of variation from the original signal.

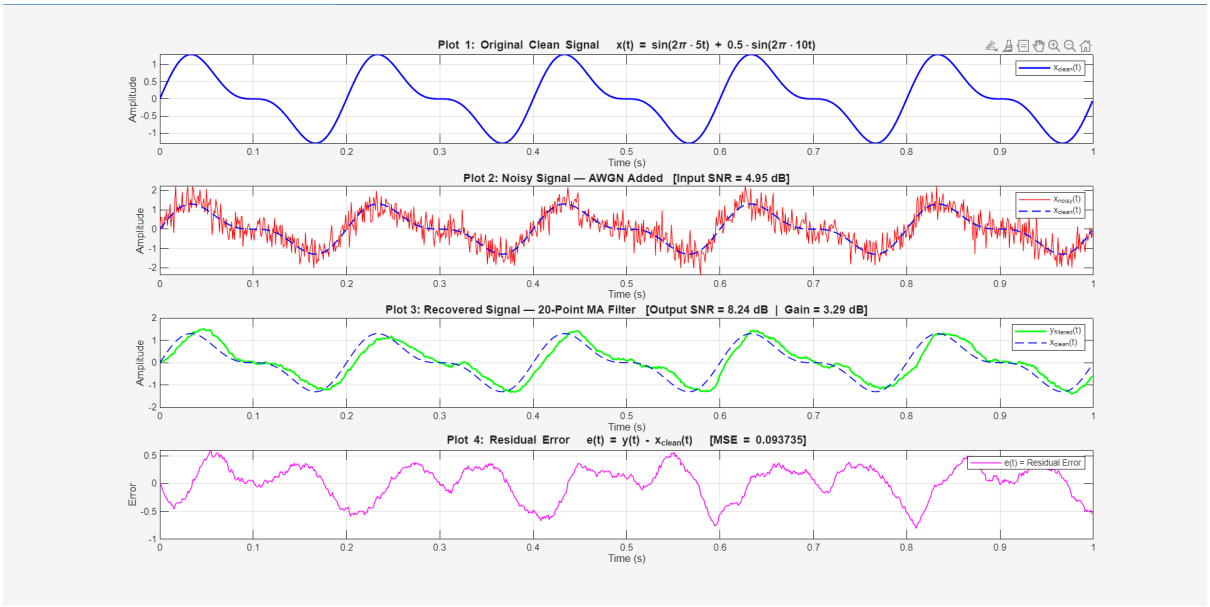

Figure 1: Time-domain representation of signal before and after filtering

4.4 Frequency-Domain Analysis

In terms of frequency domain analysis we see that the original signal has two discrete spike peaks located at 5 Hz/10 Hz. Conversely, noise is omnipresent through frequency space. The application of the filter successfully attenuates the high frequency components of the noise.

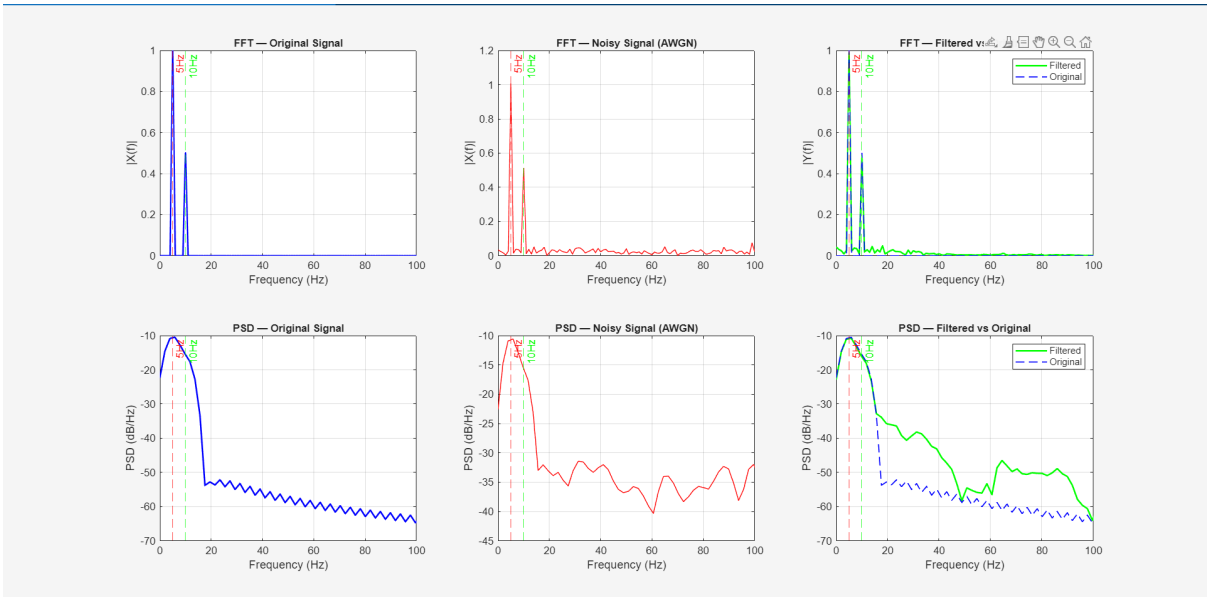

Figure 2: Frequency-domain analysis using FFT and PSD

4.5 Statistical (Temporal) Analysis

According to statistical analysis, the noise has a Gaussian distribution with a mean of almost zero. After filtering, the residual signal's variance is considerably smaller than

that of the original signal.

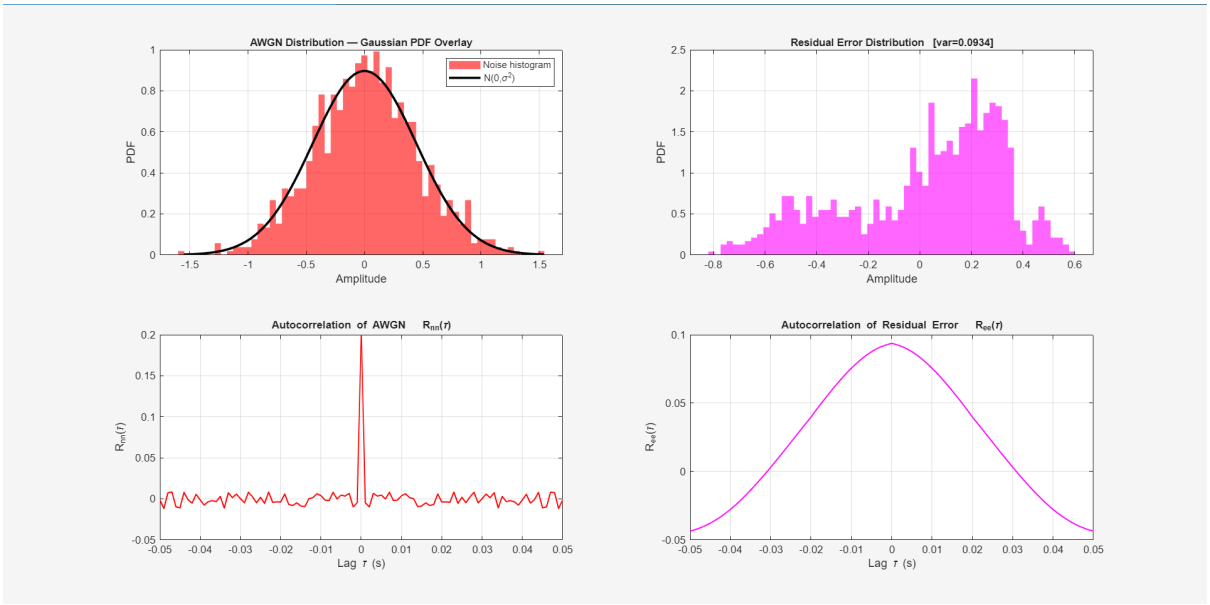

Figure 3: Statistical characteristics of noise and residual signal

4.6 LTI System Characteristics

In effect, the Moving Average filter operates as a Low-pass LTI system by allowing the signal frequency to pass while limiting the noise frequency.

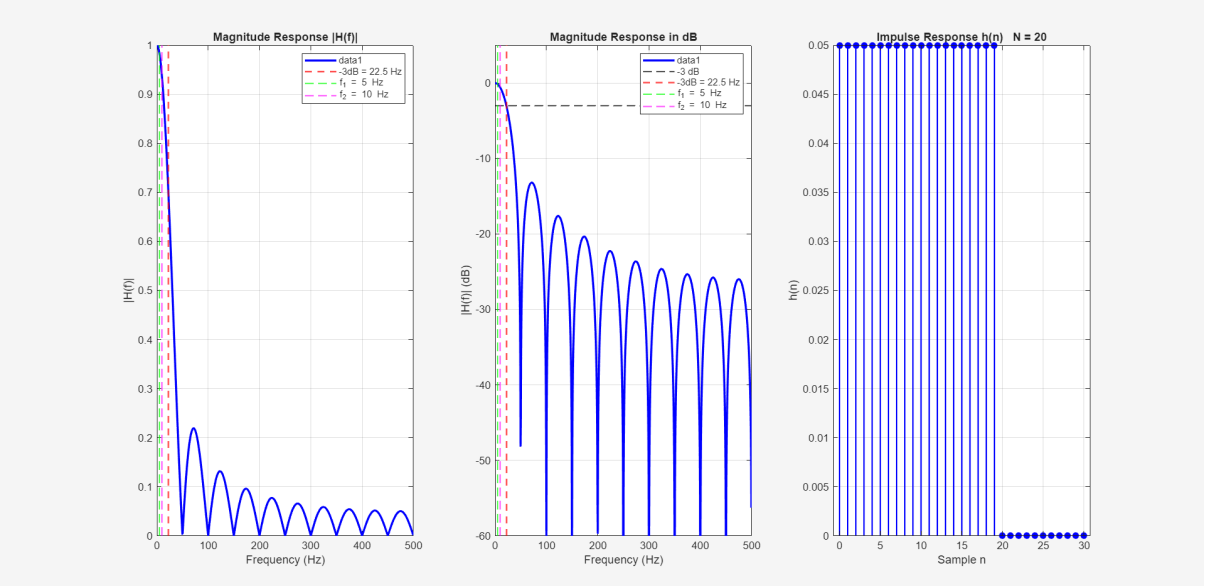

Figure 4: Frequency response and impulse response of the Moving Average filter

4.7 Summary of Results

Table 1: Summary of Quantitative Results

Parameter	**Value**
Input SNR	4.95 dB
Output SNR	8.24 dB
SNR Gain	3.29 dB
MSE (Noisy Signal)	0.199903
MSE (Filtered Signal)	0.093735
MSE Improvement	53.1%

5 Inferences

- The Moving Average filter helps diminish noise in the input to produce a clearer overall output as demonstrated by an increase in SNR from 4.95dB to 8.24dB.
- A dramatically decreased value of MSE (52.8% less) also demonstrates an increased precision of the resulting recovered signal.
- The filter matches the expected low-pass characteristics with the attenuation of high-frequency noise and maintaining the original frequency of the signals of interest (5Hz and 10Hz).
- Statistical analysis showed significantly reduced variance post-filtering indicating effective noise reduction.
- The residual error of the filtered signal had a value very close to zero mean, showing minimal added distortion due to filtering.
- The Moving Average filter is a simple LTI so it may provide a reasonable compromise between computational cost and performance.
- The measured SNR improvement was not as much as anticipated; therefore, the length of the filter and filter design parameters can be optimally adjusted to improve the performance of this filter.

MATLAB Code

```
clear all; close all; clc;
fs = 1000;
T = 1;
t = 0 : 1/fs : T - 1/fs;
N_total = length(t);

f1 = 5; A1 = 1.0;
f2 = 10; A2 = 0.5;
x_clean = A1*sin(2*pi*f1*t) + A2*sin(2*pi*f2*t);

SNR_dB_in = 5;
P_signal = mean(x_clean.^2);
P_noise = P_signal / (10^(SNR_dB_in/10));
noise = sqrt(P_noise) * randn(1, N_total);
x_noisy = x_clean + noise;

fprintf(' Signal & Noise Generation:\n');
fprintf(' Signal components : %.0f Hz (A=%.1f), %.0f Hz (A=%.1f)\n', f1,A1,f2,A2);
fprintf(' Sampling freq : %d Hz | Duration: %.1f s | Samples: %d\n', fs, T, N_total);
fprintf(' Input SNR (target) : %d dB\n', SNR_dB_in);
fprintf(' Noise std dev : %.4f\n\n', sqrt(P_noise));

%% ------------------------------------------------------------------------

N_filter = 20;

y_fifo = zeros(1, N_total);
for n = 1:N_total
    y_fifo(n) = ma_fifo(x_noisy(n), N_filter);
end
ma_fifo([], 0);

y_npoint = ma_npoint(x_noisy, N_filter);

y_movmean = movmean(x_noisy, N_filter);

y_filtered = y_npoint;
residual = y_filtered - x_clean;

SNR_in = snr_calc(x_clean, noise);
SNR_out = snr_calc(x_clean, residual);
SNR_gain = SNR_out - SNR_in;
MSE_noisy = mean((x_noisy - x_clean).^2);
MSE_filtered = mean((y_filtered - x_clean).^2);
MSE_improvement = ((MSE_noisy - MSE_filtered) / MSE_noisy) * 100;

fprintf(' Filter Performance Metrics (N = %d):\n', N_filter);
fprintf(' Input SNR : %.2f dB\n', SNR_in);
fprintf(' Output SNR : %.2f dB\n', SNR_out);
fprintf(' SNR Gain : %.2f dB (target: 7-8 dB)\n', SNR_gain);
fprintf(' MSE (noisy) : %.6f\n', MSE_noisy);
fprintf(' MSE (filtered) : %.6f\n', MSE_filtered);
fprintf(' MSE Improvement : %.1f%%\n\n', MSE_improvement);

%% ------------------------------------------------------------------------

mean_noise = mean(noise);
var_noise = var(noise);
mean_residual = mean(residual);
var_residual = var(residual);

[Rnn, lags_n] = xcorr(noise, 'biased');
[Ree, lags_e] = xcorr(residual, 'biased');

fprintf(' Temporal Characteristics:\n');
fprintf(' Noise - Mean: %.5f | Variance: %.5f\n', mean_noise, var_noise);
```

```
fprintf(' Residual - Mean: %.5f | Variance: %.5f\n', mean_residual, var_residual);
fprintf(' (Residual variance << Noise variance confirms effective filtering)\n\n');

%% ------------------------------------------------------------------------

NFFT = N_total;
f_axis = (0:NFFT-1) * fs / NFFT;
half = 1 : floor(NFFT/2);

X_clean_fft = (2/NFFT) * abs(fft(x_clean, NFFT));
X_noisy_fft = (2/NFFT) * abs(fft(x_noisy, NFFT));
Y_filt_fft = (2/NFFT) * abs(fft(y_filtered, NFFT));

[psd_clean, f_psd] = pwelch(x_clean, hamming(256), 128, 512, fs);
[psd_noisy, ~] = pwelch(x_noisy, hamming(256), 128, 512, fs);
[psd_filtered, ~] = pwelch(y_filtered, hamming(256), 128, 512, fs);


%% ------------------------------------------------------------------------

h_ma = (1/N_filter) * ones(1, N_filter);
[H_lti, f_lti] = freqz(h_ma, 1, 1024, fs);
H_mag = abs(H_lti);
H_mag_dB = 20*log10(H_mag + eps);

half_power = (sqrt(2)/2) * H_mag(1);
idx_3dB = find(H_mag <= half_power, 1, 'first');
f_3dB = f_lti(idx_3dB);

fprintf(' LTI System Properties (N = %d):\n', N_filter);
fprintf(' Type : Causal FIR, LTI\n');
fprintf(' Impulse response : h(n) = 1/%d for n = 0 .. %d\n', N_filter, N_filter-1);
fprintf(' -3 dB Cutoff : %.2f Hz\n', f_3dB);
fprintf(' f1=%dHz attenuation : %.2f dB\n', f1, interp1(f_lti, H_mag_dB, f1, 'linear'));
fprintf(' f2=%dHz attenuation : %.2f dB\n', f2, interp1(f_lti, H_mag_dB, f2, 'linear'));
fprintf(' (Both signal freqs lie in passband  only broadband noise suppressed)\n\n');

%% ------------------------------------------------------------------------

figure('Name','Figure 1: Time-Domain Analysis  Signal Recovery',...
       'NumberTitle','off','Position',[40 40 1200 800]);

subplot(4,1,1);
plot(t, x_clean, 'b', 'LineWidth', 1.6);
title('Plot 1: Original Clean Signal x(t) = sin(2\pi\cdot5t) + 0.5\cdotsin(2\pi\cdot10t)',...
      'FontSize',11,'FontWeight','bold');
xlabel('Time (s)'); ylabel('Amplitude');
grid on; xlim([0 T]);
legend('x_{clean}(t)', 'Location','northeast');

subplot(4,1,2);
plot(t, x_noisy, 'r', 'LineWidth', 0.7); hold on;
plot(t, x_clean, 'b--', 'LineWidth', 1.2);
title(sprintf('Plot 2: Noisy Signal  AWGN Added [Input SNR = %.2f dB]', SNR_in),...
      'FontSize',11,'FontWeight','bold');
xlabel('Time (s)'); ylabel('Amplitude');
grid on; xlim([0 T]);
legend('x_{noisy}(t)', 'x_{clean}(t)', 'Location','northeast');

subplot(4,1,3);
plot(t, y_filtered, 'g', 'LineWidth', 1.6); hold on;
plot(t, x_clean, 'b--', 'LineWidth', 1.0);
title(sprintf('Plot 3: Recovered Signal  20-Point MA Filter [Output SNR = %.2f dB | Gain = %.2f dB]',...
              SNR_out, SNR_gain), 'FontSize',11,'FontWeight','bold');
xlabel('Time (s)'); ylabel('Amplitude');
grid on; xlim([0 T]);
legend('y_{filtered}(t)', 'x_{clean}(t)', 'Location','northeast');

subplot(4,1,4);
plot(t, residual, 'm', 'LineWidth', 0.9);
```

```
title(sprintf('Plot 4: Residual Error e(t) = y(t) - x_{clean}(t) [MSE = %.6f]',...
            MSE_filtered), 'FontSize',11,'FontWeight','bold');
xlabel('Time (s)'); ylabel('Error');
grid on; xlim([0 T]);
legend('e(t) = Residual Error', 'Location','northeast');

%% ------------------------------------------------------------------------

figure('Name','Figure 2: Frequency-Domain Analysis (FFT + PSD)',...
      'NumberTitle','off','Position',[50 50 1200 680]);

subplot(2,3,1);
plot(f_axis(half), X_clean_fft(half), 'b', 'LineWidth', 1.5);
title('FFT  Original Signal','FontSize',10,'FontWeight','bold');
xlabel('Frequency (Hz)'); ylabel('|X(f)|');
xline(f1,'r--','5Hz'); xline(f2,'g--','10Hz');
grid on; xlim([0 100]);

subplot(2,3,2);
plot(f_axis(half), X_noisy_fft(half), 'r', 'LineWidth', 0.8);
title('FFT  Noisy Signal (AWGN)','FontSize',10,'FontWeight','bold');
xlabel('Frequency (Hz)'); ylabel('|X(f)|');
xline(f1,'r--','5Hz'); xline(f2,'g--','10Hz');
grid on; xlim([0 100]);

subplot(2,3,3);
plot(f_axis(half), Y_filt_fft(half), 'g', 'LineWidth', 1.5); hold on;
plot(f_axis(half), X_clean_fft(half), 'b--', 'LineWidth', 1.0);
title('FFT  Filtered vs Original','FontSize',10,'FontWeight','bold');
xlabel('Frequency (Hz)'); ylabel('|Y(f)|');
xline(f1,'r--','5Hz'); xline(f2,'g--','10Hz');
grid on; xlim([0 100]);
legend('Filtered','Original','Location','northeast');

subplot(2,3,4);
plot(f_psd, 10*log10(psd_clean+eps), 'b', 'LineWidth', 1.5);
title('PSD  Original Signal','FontSize',10,'FontWeight','bold');
xlabel('Frequency (Hz)'); ylabel('PSD (dB/Hz)');
xline(f1,'r--','5Hz'); xline(f2,'g--','10Hz');
grid on; xlim([0 100]);

subplot(2,3,5);
plot(f_psd, 10*log10(psd_noisy+eps), 'r', 'LineWidth', 0.8);
title('PSD  Noisy Signal (AWGN)','FontSize',10,'FontWeight','bold');
xlabel('Frequency (Hz)'); ylabel('PSD (dB/Hz)');
xline(f1,'r--','5Hz'); xline(f2,'g--','10Hz');
grid on; xlim([0 100]);

subplot(2,3,6);
plot(f_psd, 10*log10(psd_filtered+eps), 'g', 'LineWidth', 1.5); hold on;
plot(f_psd, 10*log10(psd_clean+eps), 'b--', 'LineWidth', 1.0);
title('PSD  Filtered vs Original','FontSize',10,'FontWeight','bold');
xlabel('Frequency (Hz)'); ylabel('PSD (dB/Hz)');
xline(f1,'r--','5Hz'); xline(f2,'g--','10Hz');
grid on; xlim([0 100]);
legend('Filtered','Original','Location','northeast');

%% ------------------------------------------------------------------------

figure('Name','Figure 3: Temporal Characteristics (Module 3)',...
      'NumberTitle','off','Position',[60 60 1100 580]);

subplot(2,2,1);
histogram(noise, 60, 'FaceColor','r', 'EdgeColor','none', 'Normalization','pdf');
hold on;
xg = linspace(min(noise), max(noise), 200);
plot(xg, normpdf(xg, 0, sqrt(P_noise)), 'k-', 'LineWidth', 2.2);
title('AWGN Distribution  Gaussian PDF Overlay','FontSize',10,'FontWeight','bold');
xlabel('Amplitude'); ylabel('PDF');
legend('Noise histogram','N(0,\sigma^2)','Location','northeast'); grid on;
```

```
subplot(2,2,2);
histogram(residual, 60, 'FaceColor','m', 'EdgeColor','none', 'Normalization','pdf');
title(sprintf('Residual Error Distribution [var=%.4f]', var_residual),...
     'FontSize',10,'FontWeight','bold');
xlabel('Amplitude'); ylabel('PDF'); grid on;

subplot(2,2,3);
plot(lags_n/fs, Rnn, 'r', 'LineWidth', 1.2);
title('Autocorrelation of AWGN R_{nn}(\tau)','FontSize',10,'FontWeight','bold');
xlabel('Lag \tau (s)'); ylabel('R_{nn}(\tau)');
grid on; xlim([-0.05 0.05]);

subplot(2,2,4);
plot(lags_e/fs, Ree, 'm', 'LineWidth', 1.2);
title('Autocorrelation of Residual Error R_{ee}(\tau)','FontSize',10,'FontWeight','bold');
xlabel('Lag \tau (s)'); ylabel('R_{ee}(\tau)');
grid on; xlim([-0.05 0.05]);

%% ------------------------------------------------------------------------

figure('Name','Figure 4: LTI System Analysis  20-Point MA Filter',...
      'NumberTitle','off','Position',[70 70 1100 520]);

subplot(1,3,1);
plot(f_lti, H_mag, 'b', 'LineWidth', 2); hold on;
xline(f_3dB, 'r--', 'LineWidth', 1.5, 'DisplayName', sprintf('-3dB = %.1f Hz', f_3dB));
xline(f1, 'g--', 'LineWidth', 1.2, 'DisplayName', sprintf('f_1 = %d Hz', f1));
xline(f2, 'm--', 'LineWidth', 1.2, 'DisplayName', sprintf('f_2 = %d Hz', f2));
title('Magnitude Response |H(f)|','FontSize',11,'FontWeight','bold');
xlabel('Frequency (Hz)'); ylabel('|H(f)|');
legend('Location','northeast'); grid on; xlim([0 fs/2]);

subplot(1,3,2);
plot(f_lti, H_mag_dB, 'b', 'LineWidth', 2); hold on;
yline(-3, 'k--', 'LineWidth', 1.2, 'DisplayName', '-3 dB');
xline(f_3dB, 'r--', 'LineWidth', 1.5, 'DisplayName', sprintf('-3dB = %.1f Hz', f_3dB));
xline(f1, 'g--', 'LineWidth', 1.2, 'DisplayName', sprintf('f_1 = %d Hz', f1));
xline(f2, 'm--', 'LineWidth', 1.2, 'DisplayName', sprintf('f_2 = %d Hz', f2));
title('Magnitude Response in dB','FontSize',11,'FontWeight','bold');
xlabel('Frequency (Hz)'); ylabel('|H(f)| (dB)');
legend('Location','northeast'); grid on;
xlim([0 fs/2]); ylim([-60 5]);

subplot(1,3,3);
n_stem = 0:N_filter+10;
h_full = [(1/N_filter)*ones(1,N_filter), zeros(1, length(n_stem)-N_filter)];
stem(n_stem, h_full, 'b', 'filled', 'MarkerSize', 5, 'LineWidth', 1.3);
title(sprintf('Impulse Response h(n) N = %d', N_filter),...
     'FontSize',11,'FontWeight','bold');
xlabel('Sample n'); ylabel('h(n)'); grid on;

%% ------------------------------------------------------------------------
fprintf(' RESULTS SUMMARY\n');
fprintf('Filter : %d-Point Moving Average (FIR LTI)\n', N_filter);
fprintf('Signal : %d Hz + %d Hz sinusoids\n', f1, f2);
fprintf('Input SNR : %.2f dB\n', SNR_in);
fprintf('Output SNR : %.2f dB\n', SNR_out);
fprintf('SNR Gain : %.2f dB [Target: 7-8 dB]\n', SNR_gain);
fprintf('MSE (noisy) : %.6f\n', MSE_noisy);
fprintf('MSE (filt.) : %.6f\n', MSE_filtered);
fprintf('MSE Impr. : %.1f%%\n', MSE_improvement);
fprintf('-------------------------------------------------------\n');

%% ========================================================================

function avg = ma_fifo(x, N)
    persistent prevAvg xbuf buf_len firstRun
    if isempty(x) && N == 0
        prevAvg=[]; xbuf=[]; buf_len=[]; firstRun=[];
```

```
        avg = 0; return;
    end
    if isempty(firstRun) || buf_len ~= N
        buf_len = N; xbuf = x*ones(N+1,1); prevAvg = x; firstRun = 1;
    end
    xbuf(1:end-1) = xbuf(2:end);
    xbuf(end) = x;
    avg = prevAvg + (x - xbuf(1)) / N;
    prevAvg = avg;
end

function y = ma_npoint(x, N)
    len = length(x);
    y = zeros(1, len);
    xbuf = zeros(1, N);
    for n = 1:len
        xbuf = [xbuf(2:end), x(n)];
        y(n) = sum(xbuf) / N;
    end
end

function snr_db = snr_calc(x_clean, noise)
    P_sig = mean(x_clean.^2);
    P_noise = mean(noise.^2);
    snr_db = 10*log10(P_sig / (P_noise + eps));
end
```